AI 기반 일정 관리 프로젝트 - DailyLine

개인 프로젝트 | Github

프로젝트 개요

프로젝트 설명

일정 생성 이후 알림 전송, 파일 처리 등 비동기 작업이 연쇄적으로 발생하는 특성을 고려하여, 안정적인 이벤트 처리와 장애 대응을 중심으로 구조를 설계한 웹 기반 개인 일정 관리 서비스입니다.

반복 일정·충돌 방지·실시간 알림·파일 첨부·AI 일정 추천 기능을 제공합니다.

제작 기간

- **MVP 기본 구현 : 25.01~25.03 (3개월)**
- **기능 고도화: 25.03~25.09 (6개월)**

기술 스택

- **Back-end:** JAVA, SpringBoot, MySQL, Redis, Kafka
- **Front-end:** React.js, Next.js, TypeScript,
- **Dev-Ops:** AWS, S3, Prometheus, Loki, Grafana, Docker, Nginx

핵심 구현

Back-End

- **Kafka + Outbox 패턴으로 비동기 이벤트 처리 및 메시지 유실 방지**
- **DLQ · Retry · eventId 멱등 처리로 중복 방지**
- **S3 Presigned URL + 고아 객체 방지 설계**
- **TestContainers 통합 테스트 (Outbox → Kafka → DB 흐름 검증)**
- **AI API 기반 일정 추천 + Redis 캐싱으로 외부 API 비용 최소화**

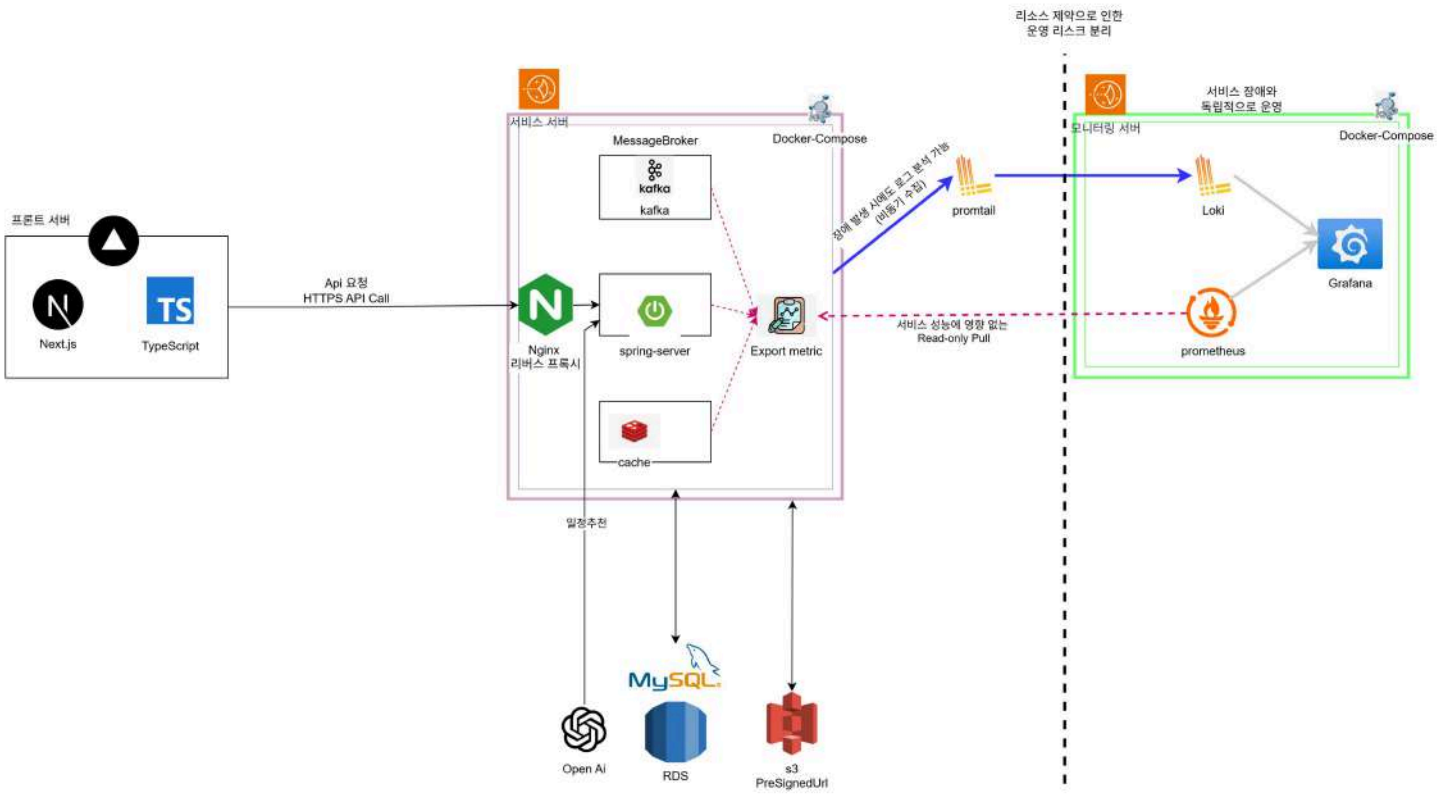
Infra

- **Prometheus + Grafana + Loki 모니터링 구축**
- **MDC 기반 비동기 로그 추적**
- **JVM 튜닝 (G1GC, Heap 최적화)**
- **JMeter 부하 테스트로 처리량 · 응답시간 수치화**

프로젝트 아키텍처

제한된 리소스 환경에서 운영 리스크를 최소화하기 위한 서버 분리 아키텍처

- 이 프로젝트는 서비스 성능에 미치는 영향을 최소화하기 위해 서비스 서버와 모니터링 서버를 물리적으로 분리하여 운영 안정성을 확보했습니다. 단순한 기능 구현을 넘어, 장애 상황에서도 관측 가능성을 유지하는 운영 중심 아키텍처를 목표로 했습니다.

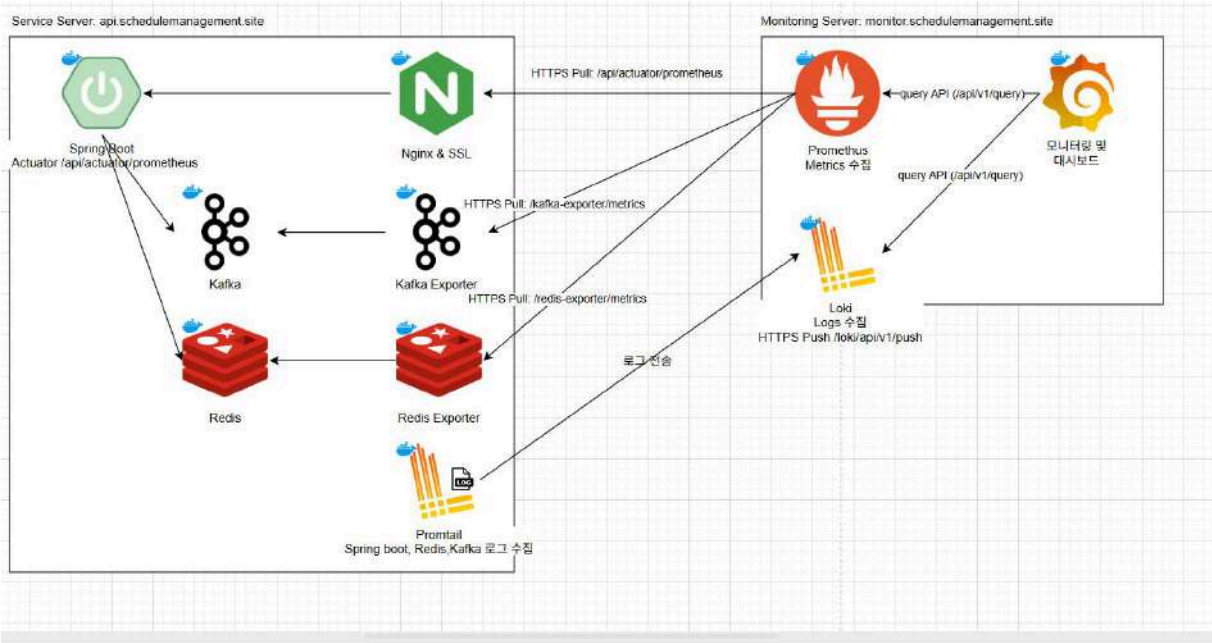


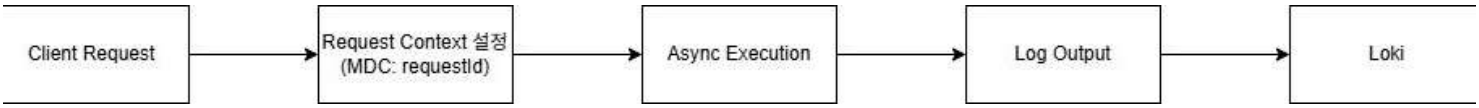
기술적 선택 이유

- 모니터링 서버 분리 서비스 성능 간섭 방지 + 장애 시 독립적 운영
- AWS Lightsail 고정 비용으로 예산 예측 + 설정 단순화
- Nginx SSL 처리 + 서버 직접 노출 방지
- Kafka 알림·파일처리 등 비동기 작업을 서비스 로직과 분리
- Redis AI 추천 결과 캐싱으로 외부 API 중복 호출 방지
- S3 Presigned URL 파일 업로드 트래픽을 서버 부하 없이 처리

모니터링 아키텍처

메트릭·로그 수집 기반 장애 원인 분석 아키텍처





모니터링 구축 이유

- Prometheus + Grafana + Loki 운영 환경에서 발생하는 장애를 신속하게 파악하기 위해 메트릭(Prometheus)·로그(Loki)를 Grafana 단일 대시보드에서 상관 분석이 가능하도록 구성
- 특정 시간대 API 지연 발생 시 동시간대 Kafka Consumer Lag·JVM Heap 상태를 함께 확인하여 원인 파악 가능

주요 관측 지표

- Application: JVM Heap 사용량, GC 발생 시간, Active Thread 수, API 요청 응답 속도(Latency)
- Message Queue: Kafka 초당 처리량, Consumer Lag(지연), DLQ(Dead Letter Queue) 재처리 발생률
- Storage & Cache: Redis 메모리 점유율, 실시간 Connection 수, 데이터 읽기/쓰기 지연 시간

분산 추적을 위한 MDC 로그

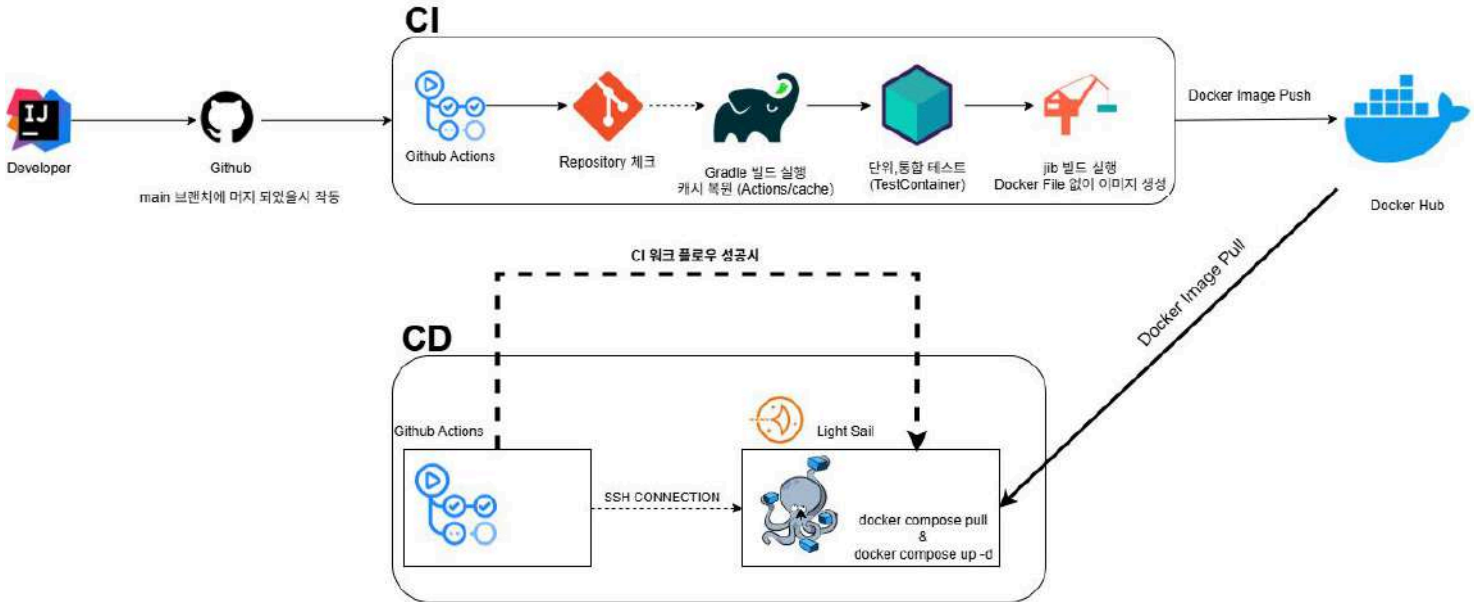
- 알림 전송,회원가입 등 비동기 처리 시 스레드 전환으로 requestId가 유실되어 어떤 요청에서 발생한 로그인지 추적 불가
- 단순 로그만으로는 Kafka 처리 중 장애 발생 시 원인 파악이 어려움
- MDC로 requestId를 비동기 스레드까지 전파하여 API 요청부터 Kafka 처리까지 하나의 흐름으로 추적

적용 범위

- 이벤트 적용 부분 (회원 가입, 일정 알림)
- 썸네일 이미지 생성부분

CI / CD 구축도

CI 워크플로우 성공 시 자동 배포되는 파이프라인



배포 효율화와 환경 일관성 유지

- 코드 변경 사항을 운영 환경에 신속하고 안전하게 반영하기 위해 GitHub Actions 기반의 완전 자동화 파이프라인을 구축했습니다. 특히 Dockerfile 없이도 최적화된 이미지를 생성하는 Jib을 도입하여 빌드 속도를 개선하고 배포 프로세스를 간소화

CI (Continuous Integration)

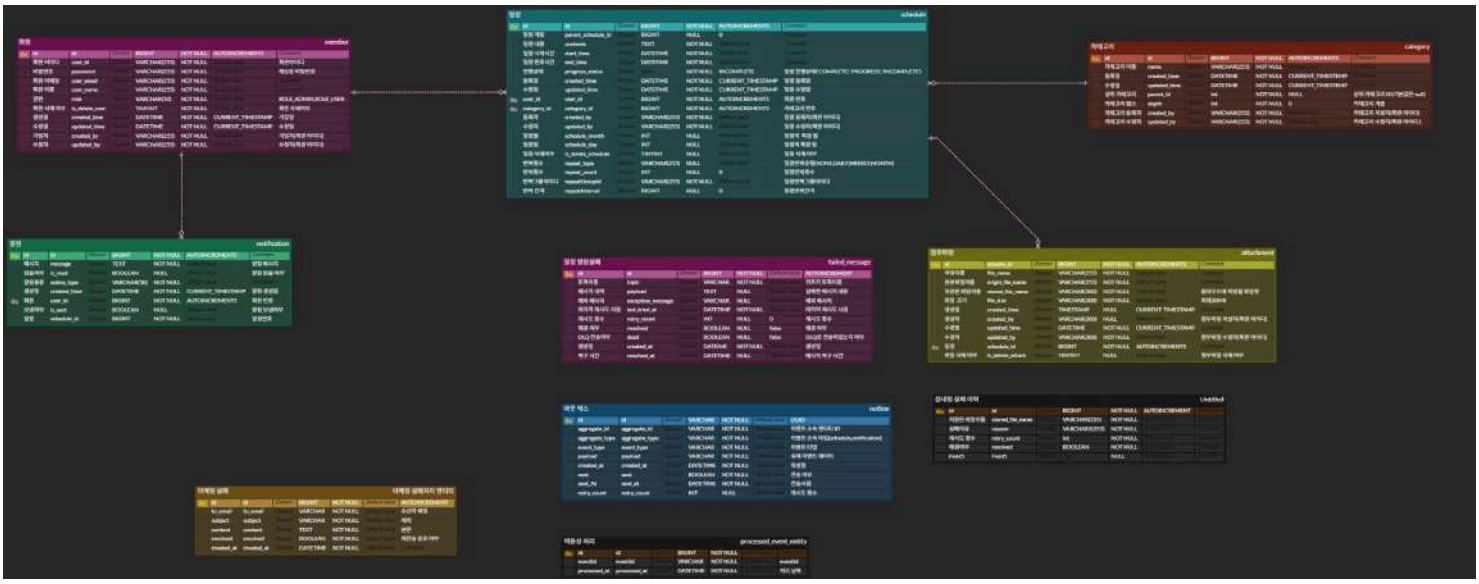
- Trigger
  - main 브랜치 Merge 시 자동 실행 및 workflow_run 트리거 활용
  - CI 성공 시에만 CD가 실행되도록 설계하여 미완성 이미지의 배포 위험을 원천 차단
- Test & Build
  - Gradle 빌드 및 Testcontainers 기반 통합 테스트 수행
  - Kafka, MySQL, Redis를 실제 컨테이너로 기동하여 DLQ·멱등성·재처리 흐름을 운영과 동일한 환경에서 검증
- Caching Strategy
  - actions/cache를 활용한 Gradle 의존성 캐싱으로 빌드 시간 약 1분 이상 단축

CD (Continuous Deployment)

- Jib 기반 이미지 빌드
  - Dockerfile 없이 레이어 단위 최적화 이미지를 생성하여 빌드 속도 향상
  - 커스텀 설정의 제약이 있으나, 표준 Spring Boot 앱 구조에서는 빌드 복잡도를 낮추는 데 유리하다고 판단하여 채택.
- SSH Remote Deployment & Security
  - AWS Lightsail 대상 SSH 원격 배포 자동화
  - --password-stdin 활용 및 GitHub Secrets 관리로 배포 로그 내 민감 정보(Password) 노출 방지
- Infrastructure as Code & Automation
  - docker-compose를 통한 선언적 인프라 관리로 개발-운영 환경 간 일관성 확보
  - pull && up -d 전략을 통해 최신 이미지 기반으로 컨테이너 교체 프로세스를 자동화하고 서비스 정지 시간(Downtime)을 최소화

ERD 설계도

데이터 정합성과 시스템 복원력 확보



객체 참조를 배제한 ID 기반 약결합

- 엔티티 간 직접적인 객체 참조 대신 ID(Long) 값만 보유하는 구조를 채택했습니다. 이를 통해 특정 도메인의 재처리 작업이 연쇄적인 조회를 일으키는 것을 방지하고, 도메인 간의 의존성을 최소화
- 조회 시 Join 쿼리가 증가하는 단점이 있지만, 도메인 격리가 더 중요하다고 판단하여 이 구조를 선택.

DB 제약조건을 통한 정합성 강제

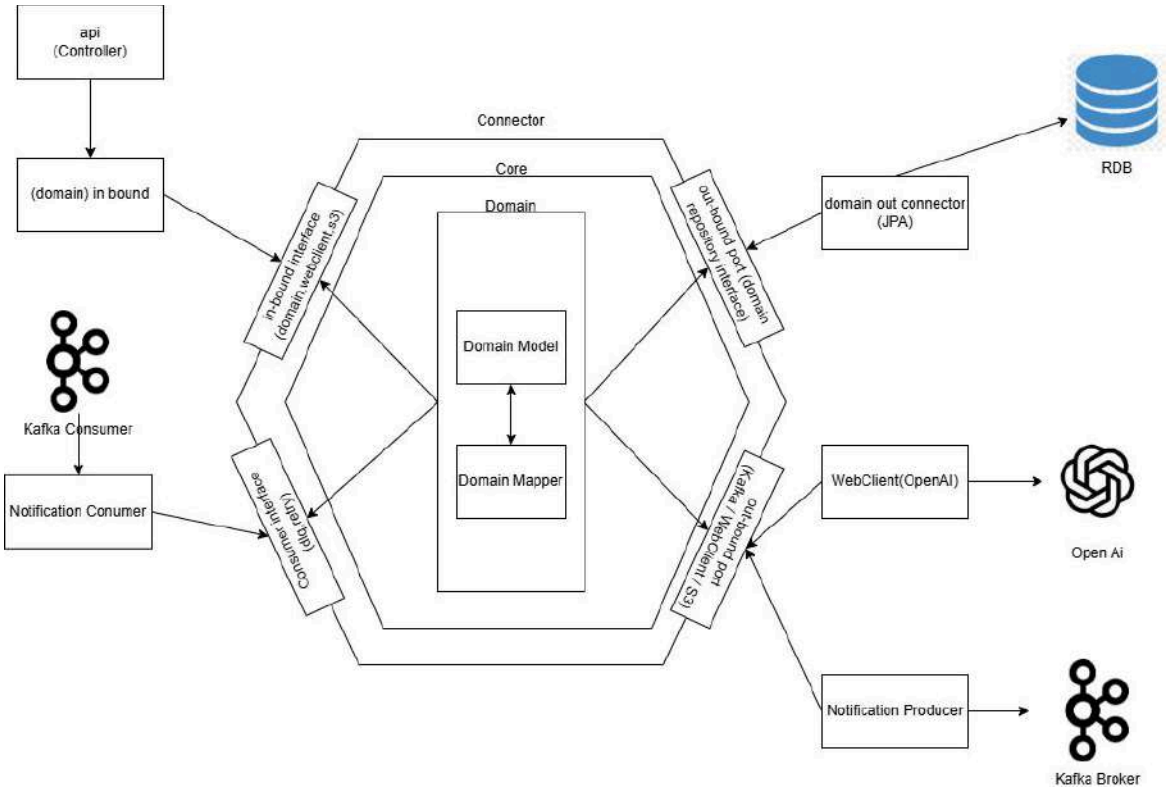
- 데이터 무결성을 애플리케이션 계층(JPA)에만 의존하지 않고, DB 수준의 Foreign Key와 제약조건으로 강제했습니다. 이는 운영 단계에서 어떤 기술적 변수가 발생하더라도 데이터의 최종 정합성을 보장.
- DB 의존성이 높아지는 단점이 있지만, 운영 단계에서 어떤 기술적 변수가 발생하더라도 최종 정합성을 보장하는 것이 더 중요하다고 판단했습니다.

이벤트 처리 인프라의 독립성

- Outbox, FailMessage(실패 기록), ProcessedEvent(멱등성 처리) 테이블을 독립적으로 설계. 특정 기능의 실패가 전체 시스템으로 전이되지 않도록 격리하여, DLQ 재처리나 EOS(Exactly-Once Semantic) 상황에서도 안정적인 복구가 가능.
- 테이블 수 증가로 관리 복잡도가 올라가는 단점이 있지만, DLQ 재처리·EOS 상황에서도 안정적인 복구가 가능한 구조가 더 중요하다고 판단.

헥사고날 아키텍처

외부 의존성을 제어하고 도메인을 격리하여 테스트 용이성까지 확보한 구조 설계



Domain & Application Core

- 외부 라이브러리 의존성 없이 순수 Java 코드로만 작성된 비즈니스 로직 계층입니다.
- Domain Model & Mapper: 시스템의 핵심 규칙을 정의하며, 인프라 계층의 엔티티와 도메인 모델을 분리하여 데이터 오염을 방지합니다. 계층 분리로 코드량이 늘어나는 단점이 있지만, 도메인 로직이 외부 기술에 오염되지 않는 것이 더 중요하다고 판단했습니다.

Inbound Adapters

- Web Controller & Kafka Consumer: 외부의 HTTP 요청이나 Kafka 이벤트를 수신하는 통로입니다.
- Consumer Interface: 특히 알림 컨슈머단계에서 DLQ 및 Retry 로직을 인터페이스화하여 메시지 처리의 안정성을 확보했습니다. 인터페이스 추가로 초기 설계 비용이 증가하지만, 구현체 교체 시 도메인 코드 변경 없이 어댑터만 수정하면 되는 구조가 더 중요하다고 판단했습니다.

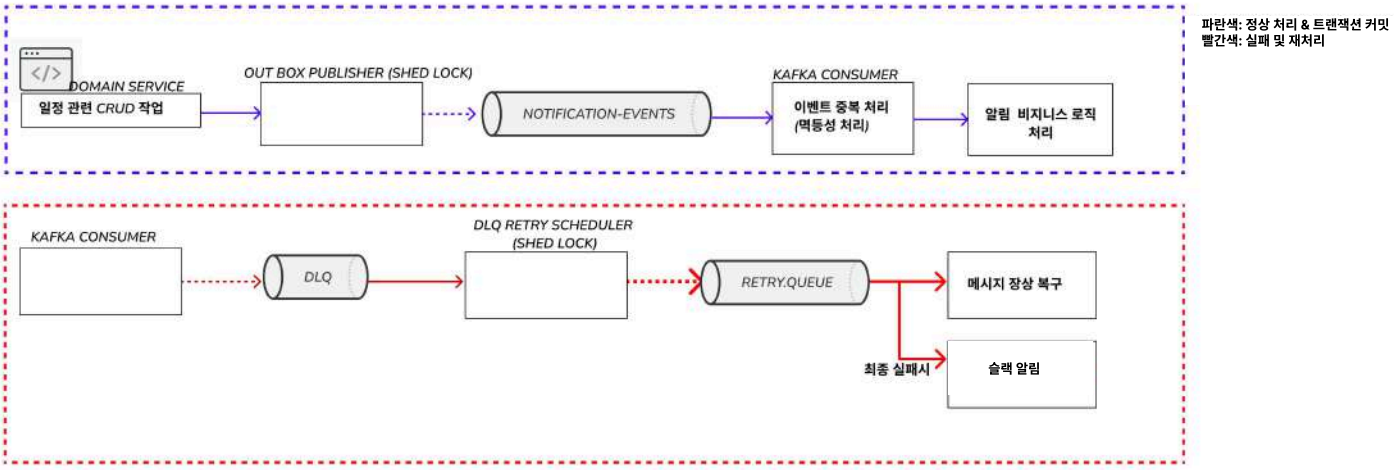
Outbound Adapters

- Persistence (JPA) & Messaging (Producer): 도메인에서 정의한 Port를 실제로 구현하여 DB 저장 및 Kafka 메시지 발행을 수행합니다.
- External API: OpenAI WebClient 등을 통해 외부 지능형 서비스와 연동하며, 이 역시 Port 뒤에 숨겨 도메인 로직과의 결합도를 낮췄습니다. 테스트 시 Mock으로 교체해 도메인 로직만 단독으로 검증할 수 있습니다.

트러블 슈팅

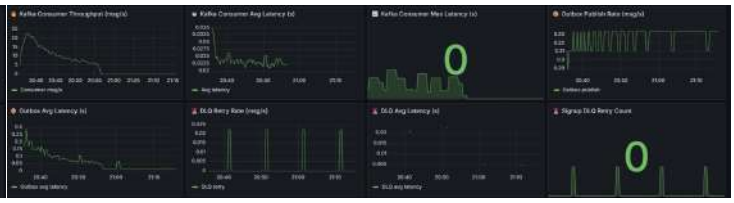
메시지 처리 신뢰성 확보 (중복·유실 방지)

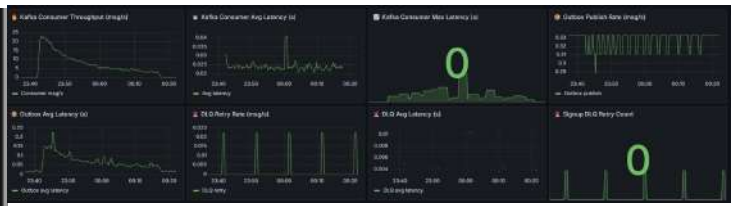
1.비즈니스 트랜잭션과 분리된 이벤트 처리로 중복·유실 없는 메시지 처리 보장



Case.A) 동시 접속 (50VU, 100VU)

Case.B) 일괄 처리 카프카 지표(3만건,5만건)





[문제]

- 일정 등록 로직과 알림 저장 로직이 하나의 트랜잭션에 묶여, 외부 요인(알림 DB 장애 등)에 의해 핵심 기능인 '일정 등록'까지 롤백되는 현상 발생
- 단순 재처리(Retry) 도입 시 알림이 중복 발송되거나, 메시지 유실 시 데이터 정합성이 깨지는 사용자 경험 저해 우려

[해결]

- 1단계: Spring 이벤트(AFTER_COMMIT) 기반 분리
  - 메인 로직 커밋 후 이벤트를 발행하여 장애 전파를 차단했으나, 커밋 직후 서버 장애 시 이벤트 발행 자체가 누락되는 '메시지 유실' 한계 확인.
- 2단계: Kafka 도입을 통한 비동기 파이프라인 구축
  - **Kafka 선택 이유** Redis Pub/Sub는 구독자 없을 시 메시지 유실, RabbitMQ는 소비 완료 후 메시지 삭제로 재처리 불가. Kafka는 Offset 기반 복구와 Consumer Group 정합성으로 장애를 '유실'이 아닌 '지연'으로 처리할 수 있어 선택했습니다. 단, **단일 브로커 구성으로 SPOF가 존재하며 프로덕션 환경에서는 3-broker 클러스터 구성이 필요합니다.**
  - 알림 처리를 완전히 비동기로 전환하고 Kafka를 도입하여 시스템 간 결합도를 낮추고 대량 처리 기반을 마련했다. 다만 이벤트 발행 시점의 신뢰성 문제는 여전히 존재했다.
- 3단계: Transactional Outbox 적용 (이벤트 방식 제거)
  - 이벤트 발행의 불확실성을 제거하기 위해 이벤트를 삭제하고, 비즈니스 로직과 알림 데이터를 하나의 로컬 트랜잭션으로 묶어 Outbox 테이블에 기록.
  - 서버 다운 시에도 DB 기록을 기반으로 재시작 시 100% 발행 보장(At-least-once Delivery) 체계 구축.
- 4단계: 실무적 멱등성 설계 (Unique Index)
  - Outbox의 재전송으로 인한 중복 수신 문제를 해결하기 위해, 컨슈머 측에 eventId + Unique 제약 조건을 적용하여 무거운 Kafka EOS 옵션 없이도 DB 레벨에서 중복 저장 차단.

[검증]

Kafka EOS 없이 Exactly-once 효과 구현 처리 구조가 동시성 및 장애 상황에서도 메시지 유실 없이, 중복 없이 처리 되는지를 수치로 검증하기 위해 부하 테스트를 수행했다.
동시 접속 테스트(Case.A) 50VU, 100VU, 일괄 테스트(Case.B) 3만건, 5만건으로 나눠서 진행.

1. 동시 접속 테스트 (Case.A: 50 VU(위), 100 VU(아래))

- 50 VU 결과: 평균 응답 속도 187ms를 기록하며, 부하 상황에서도 지연 없는 사용자 경험을 확인했습니다.
- 100 VU 결과: 요청 수가 2배 증가했음에도 Error Rate 0.00%를 유지하여 멱등성 설계의 완결성을 증명했습니다.

Case A 환경에서 메시지 유실 0건, 중복 처리율 0%를 확인했다.

2. 대량 일괄 처리 테스트 (Case.B: 3만 건(위), 5만 건(아래))

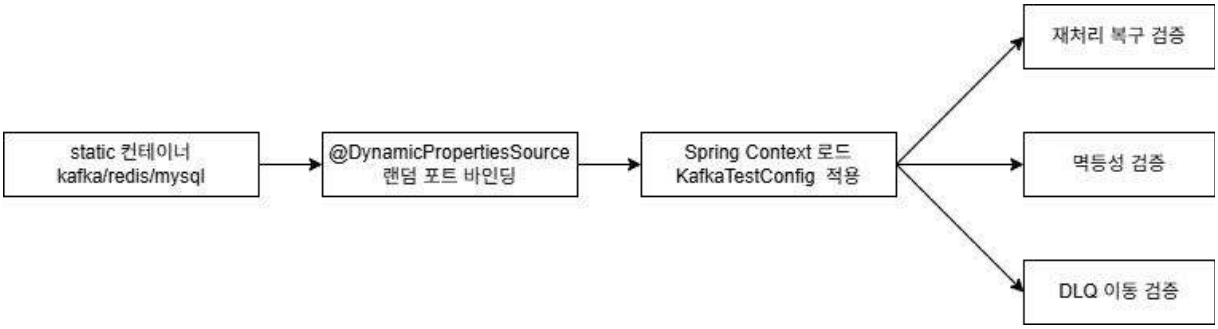
- **3만 건 결과** : 대량 유입 시 초당 처리량이 피크를 기록한 후, Consumer Lag이 정체 없이 즉각적으로 해소되는 '역V자형 처리 곡선'을 확인했습니다.
- **5만 건 결과** : Outbox Publish Rate가 약 0.33 msg/s의 일정한 주기를 그리며 끊김 없이 발행되는 것을 확인했습니다. 이는 ShedLock 기반 스케줄러가 5만 건의 대규모 데이터를 누락 없이 Kafka 브로커로 안정적으로 전달하고 있음을 입증하는 핵심 지표입니다.
- **복원력 및 정합성 증명** : Kafka Consumer Throughput이 최대 23 msg/s까지 수직 상승한 뒤, 처리가 완료됨에 따라 0으로 수렴하는 추이를 확인했습니다. 특히 Signup DLQ Retry Count가 '0'으로 유지된 점과 Scheduler Execution Rate가 안정적인 파형을 유지한 점을 통해, 대량 부하 환경에서도 데이터 유실이나 중복 없이 원자적 처리가 완료되었음을 수치로 입증했습니다.

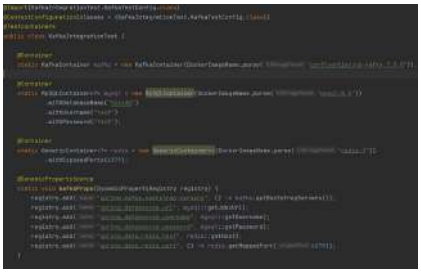
Case B 동시성 환경에서도 Consumer Lag 증가 없이 Exactly-once 처리가 유지되었다.

[성과]

- 동시성 및 장애 상황에서도 메시지 유실 0건, 중복 처리율 0%를 수치로 검증
- Kafka Exactly-once semantics를 Kafka 설정에 의존하지 않고, 애플리케이션 레벨의 멱등성(eventId 기반)과 DB 제약조건으로 보장
- Grafana 기반 메트릭으로 처리량, Error Rate, Consumer Lag을 실시간 관측 가능

2.외부 의존성을 포함한 통합 테스트 환경 구축





[문제]

- Kafka, Redis, MySQL 등 외부 인프라가 엮인 이벤트 처리 구조는 Mock 기반의 단위 테스트만으로는 메시지 유실, DLQ 이동, 재처리 흐름을 실제와 동일하게 검증하기 어려움

[해결]

- 1단계: Testcontainers 환경 구성
  - Testcontainers를 활용해 Kafka, Redis, MySQL을 독립 컨테이너로 구동
  - @DynamicPropertySource로 랜덤 포트를 동적 바인딩하여 CI/CD 환경에서도 인프라 의존성 없이 운영과 동일한 테스트 환경을 재현함
  - static 컨테이너 선언을 통해 단일 테스트 클래스 내의 모든 메서드가 인프라를 재사용하도록 설계했습니다. 이를 통해 테스트 메서드마다 컨테이너를 새로 생성하는 리소스 낭비를 차단하고 테스트 수행 속도를 확보

- 2단계: 멱등성 검증 (Exactly-Once 보장)
  - Producer의 acks=all과 enable.idempotence=true 설정으로 인해 발생하는 처리량(Throughput) 저하와 지연 시간(Latency)을 감수하더라도, 데이터 무결성을 최우선으로 선택
  -
  - 동일 eventId를 가진 이벤트가 중복 발행되더라도 단 한 번만 처리되는 멱등성을 검증
  - Exactly-once 처리가 설정이 아닌 로직과 데이터 기준(eventId)으로 보장됨을 증명

- 3단계: DLQ → 재처리 → 복구 검증
  - Consumer 처리 실패 시 이벤트가 DLQ로 이동하고, 재처리 스케줄러를 통해 정상 상태로 복구되는 흐름을 검증
  - 장애 상황에서도 메시지 유실 없이 복구 가능한 구조임을 테스트로 증명

[성과]

- **운영 수준의 Kafka 알림 및 DLQ 복원력을 테스트 환경에서 검증.**
- **그 결과 메시지 유실률은 0%로 보장되었고, 테스트 자동화를 통해 신뢰성 있는 배포 프로세스를 확립.**
- **장애 상황을 배포 전에 차단할 수 있어 사용자에게 안정적인 알림 경험을 제공했고, 서비스 신뢰도를 한층 강화.**

3.운영 환경 장애 재현과 JVM 튜닝을 통한 안정화

Before

After

Result

[문제]

- 프로젝트를 Lightsail 2GB 인스턴스에서 운영하며 Docker Compose로 서비스 서버와 모니터링 스택을 함께 구성
- 단일 서버에서 동시에 실행하자 메모리 사용량이 1.5GB 이상까지 상승했고, 모니터링 스택 활성화 시 메모리 과부하로 서버가 다운되는 문제가 발생

[해결]

- 1단계: 모니터링 서버로 분리
  - Grafana, Prometheus, Loki를 별도의 Lightsail 인스턴스로 분리하고 서비스 서버에는 Promtail과 Exporter만 유지하여 서비스 로직과 관측 로직 간 리소스 경쟁을 제거

- 2단계: 로그 관리 자동화
  - Promtail이 Docker Volume 로그를 지속 수집하며 디스크 I/O 병목을 유발하고 있었기 때문에 logrotate를 적용해 일별 순환, 압축, 자동 삭제 정책을 구성

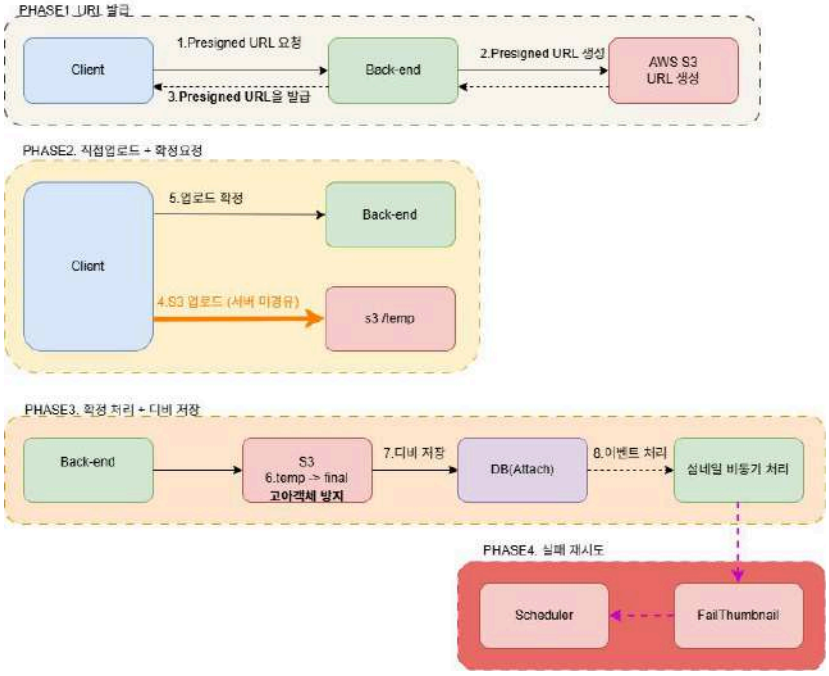
- 3단계: JVM 성능 튜닝 (모니터링 분리 이후)
  - 모니터링 서버 분리 이후(Before)에도 순수 애플리케이션 영역에서 Heap 사용량 변동 폭이 크고, Full GC 스파이크가 반복적으로 발생했다.
  - 서버 다운은 발생하지 않았으나, GC Pause Time이 0.1s 이상 반복적으로 관측되어 장기 운영 관점에서 불안정하다고 판단했다. 이에 따라 GC 로그 및 Grafana 메트릭을 기반으로 다음과 같은 JVM 튜닝을 수행했다.
  - G1GC 적용으로 GC Pause 분산, Xms / Xmx 조정으로 Heap 확장·축소 비용 제거, Metaspace 및 DirectMemory 상한 설정으로 메모리 사용 범위 명확화

[성과]

- CPU 사용률
  - 튜닝 전: 1~3% 구간에서 잦은 스파이크
  - 튜닝 후: 0.5~1.5%로 안정화
- Thread 수
  - 튜닝 전: 급격한 감소 구간 발생
  - 튜닝 후: 73~75개로 안정적 유지
- GC Pause Time
  - 튜닝 전: 0.1초 이상 스파이크 반복
  - 튜닝 후: 0.01초 이하로 안정화

- **최대 Heap 사용량 1.6GB → 1.4GB로 감소**

4.비동기 파일 업로드 환경에서의 리소스 정합성 문제 해결



[문제]

- 기존 multipart 방식은 서버가 직접 S3에 업로드하는 구조로, 파일 업로드 시 서버 I/O와 스레드를 점유했습니다.
- Presigned URL을 도입했지만, 클라이언트가 업로드 후 등록 API를 호출하지 않으면 S3에 고아 객체가 누적
- 단순 스캔 후 삭제 방식은 운영 파일과 임시 파일을 구분하기 어려워 정상 데이터를 삭제할 위험이 있었다.

[해결]

- 1단계: Presigned URL을 temp/{uuid} 경로로만 발급. 서버 승인 전까지 임시 객체로 취급, 운영 경로와 물리적으로 분리
- 2단계: POST /complete-upload 호출 시 copy 성공 확인 → 원본 delete 순서 적용 — 복사 실패 시 원본 유지로 데이터 유실 방지
- 3단계: temp/ 에 S3 Lifecycle Rule 적용. 이탈·네트워크 중단 등 서버 로직으로 커버 불가한 예외를 2단계와 독립적으로 처리하는 안전망
- 4단계: 썸네일 실패 시 즉시 예외 대신 FailThumbnail 테이블에 기록 → 5분 주기 스케줄러가 retry 3회 이하 항목 최대 20건 재처리 → 성공 시 resolved 처리

[검증]

AWS Lightsail 2vCPU/2GB 9.83KB PNG 각 5회 반복 Micrometer @Timed + Prometheus/Grafana

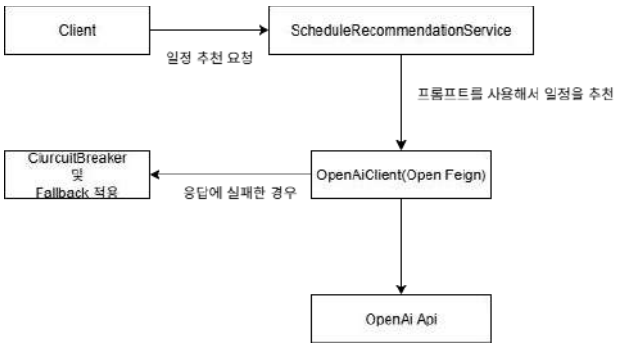
- **처리량: 0.006 → 0.016 req/s (▲2.6배) — 서버 I/O 제거**
- **평균 응답시간: 0.10s → 0.29s — 서버 부하 클라이언트 분산 (트레이드오프)**
- **P95: 0.10s → 1.20s — 클라이언트 네트워크 변동 영역**

[성과]

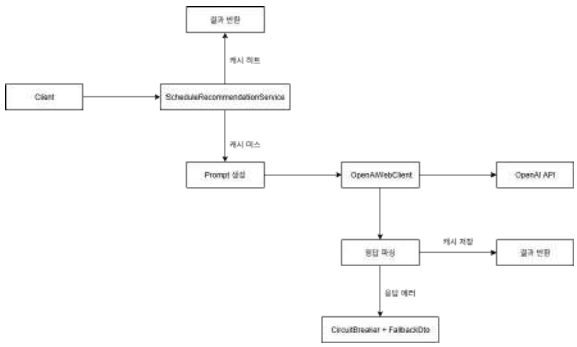
- **서버 처리량 2.6배 향상 (0.006 → 0.016 req/s) — 파일 전송 I/O가 서버에서 완전 제거**
- **응답시간 증가(0.1s → 0.29s)는 서버 부하를 클라이언트/S3로 분산한 트레이드오프 동시 업로드 요청이 늘어도 서버 자원은 안정적으로 유지되는 구조 확보**
- **temp/final 2단계 경로 분리 + S3 Lifecycle Rule 이중 안전망으로 고아 객체 문제 구조적 해결**
- **썸네일 실패 시 최대 3회 자동 재처리로 데이터 정합성 유지**

5.외부 API 의존 환경에서의 응답 지연 및 병목 개선

Before



After



(A) 50VU

(B)100VU

[문제]

- 일정 추천 기능의 외부 API(OpenAI) 연동을 OpenFeign 기반 동기 호출로 구현
- 외부 API 응답 대기 동안 요청 스레드가 블로킹되어, 동시 요청 증가 시 서버 스레드 풀 점유 시간 증가
- 개발 편의성은 높았으나 운영 환경에서 서버 자원을 통제하기 어려운 구조

[해결]

외부 API 호출로 인한 병목을 제거하기 위해 WebClient 기반 비동기 구조로 전환하고, 캐싱을 함께 적용했다.

- 1단계: WebClient 비동기·논블로킹 전환 — 외부 API 대기 중에도 요청 스레드 즉시 반환
- 2단계: 외부 API 클라이언트 증가에 대비해 공통 WebClient 추상화 (Timeout·예외처리·로깅 일원화)
- 3단계: Redis 캐싱 적용 — 동일 요청 시 외부 API 재호출 제거 (TTL 6시간)
- 4단계: CircuitBreaker 기반 Fallback 구조 — 외부 API 장애 시 서비스 중단 방지

[검증]

일정 추천이 잘 구현이 되는지 그리고 캐싱이 되는지를 확인을 해보고 얼마나 버틸수 있는지를 부하 테스트(50VU,100VU)를 진행

- 50VU (Ramp-up 30s, Duration 3min): 평균 346ms / P95 556ms / P99 709ms / 에러율 0% / Heap 118MiB 안정
- 100VU (Ramp-up 30s, Duration 3min): 평균 945ms / P95 1.78s / P99 2.1s / 에러율 0% / Heap 안정 유지

[성과]

- **동기 Feign 구조에서 WebClient 기반 비동기 처리로 전환해 스레드 점유 제거**
- **추천 결과 캐싱으로 외부 API 중복 호출 차단**
- **부하 증가 상황에서도 일정 추천 응답 시간 안정성 유지**
- **외부 API 응답 지연·장애가 서버 처리 흐름에 영향을 주지 않는 구조 확보**

회고

이 프로젝트는 처음에는 개인 일정 관리를 위한 간단한 웹 서비스를 목표로 시작했으나, 실제 구현 과정에서 CRUD 중심의 기능 구현만으로는 일정 관리 도메인이 구조적으로 복잡하다는 점을 확인했다. 반복 일정, 충돌 감지, 수정 전파와 같은 요구사항은 설계 방식에 따라 코드의 유지 가능성을 크게 좌우했다. 이에 따라 도메인 중심 설계를 기준으로 책임을 재정의하고, 외부 의존성과 인프라 요소를 분리하기 위해 헥사고날 아키텍처 기반으로 전체 구조를 재정비했다.

프로젝트의 전환점은 비동기 기능을 도입하면서 발생했다. 알림 발송, 파일 업로드, 일정 추천과 같이 외부 의존성과 처리 시간이 불확실한 기능들이 기존 트랜잭션 구조를 쉽게 무너뜨렸고, 작은 실패가 전체 요청 롤백으로 이어지는 문제가 반복적으로 발생했다. 이를 해결하기 위해 트랜잭션과 메시징을 분리하는 Outbox 패턴을 적용했고, 장애를 흡수하기 위한 DLQ·Retry 구조와 eventId 기반 멱등 처리로 중복과 유실을 제어했다.

설계한 구조가 실제 운영 환경에서도 유효한지를 확인하기 위해 Kafka, DB, Redis를 포함한 통합 테스트 환경을 구성하고, Grafana 기반 모니터링을 통해 처리량, 지연, 실패율을 수치로 검증했다. 이를 통해 비동기 메시징 구조와 멱등 처리, DLQ 기반 복구 전략이 장애 상황에서도 메시지 유실 없이 동작함을 확인했고, 장애 대응은 사후 조치가 아니라 사전 감지·격리·복구가 가능한 구조 설계의 문제임을 검증했다.

다만 현재 단일 브로커 구성으로 SPOF가 존재한다는 한계를 인식하고 있으며, 프로덕션 환경에서는 3-broker 클러스터 구성이 필요하다는 점을 과제로 남겼다.